

The background features a dark blue gradient with three interlocking gears. A large, light blue gear is centered behind the title. Two smaller, darker blue gears are positioned at the top right and bottom left. A thick, dark brown metal chain is draped around the gears, forming a circular shape that frames the central text.

API Design—Wesley Beary

Arquitectura del Software

Julián Fernández Herruzo

UO300199@uniovi.es

Miguel Arias Guerrero

UO288285@uniovi.es

Hugo Carbajales Quintana

UO300051@uniovi.es

Wesley Beary y el Diseño Empático

- ¿Application Programming Interface?

Assumptions Probably Incorrect

Las suposiciones fallan

- El diseño abstracto no sobrevive al mundo real
- Solución: Prototipos rápidos y enfoque iterativo.



Pruebas a Ciegas ("Blind Play Test")

- El sesgo del desarrollador: Estar "demasiado cerca del problema".
- La técnica: Entregar la documentación y observar en silencio.
- La regla de oro: Si intervienes para ayudar, el test fracasa.

"Intentar crear algo que sea bueno para todos, a menudo termina siendo confuso para todos."



El Principio de Menor Sorpresa

- La consistencia genera confianza: Si aprendes una regla, debería aplicar a todo el sistema.
- El caos del vocabulario: ¿Para borrar usamos delete, remove o destroy?
- El objetivo: Caminos predecibles y cero errores crípticos ("Algo salió mal").



CRUD Sobrecargado

✗ Problema: update sobrecargado

```
PUT /servers/123  
{ "state": "restarting" }
```

¿Qué hace realmente?

- ¿Actualiza un campo?
- ¿Arranca un proceso?
- ¿Envía notificaciones?
- "Depende..."



Separar estado de las acciones

✗ Sin separación clara:

`/servers/123/restart`

✓ Con namespace `/actions/`:

`/servers/123/actions/restart`

 Documentación clara

Las acciones y los recursos tienen reglas distintas y documentadas por separado

 Sin colisión de nombres

El namespace evita conflictos con sub-recursos futuros

CLI vs API Web: ¿Son lo mismo?



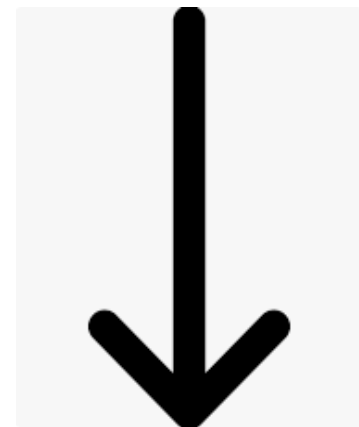
API Web General

- CRUD estándar sobre recursos
- Endpoints documentados públicamente
- Usada por múltiples clientes

CLI

- Orquesta múltiples llamadas API
- Puede tener endpoints propios especializados
- Flujos de trabajo complejos → un comando

Empujar la complejidad hacia abajo



ANTES — CLI realizaba 4 llamadas

```
anchor lcl setup
```

→ llamada 1: crear certificado

→ llamada 2: instalar

→ llamada 3: configurar servidor

→ llamada 4: verificar estado



DESPUÉS — Un endpoint de workflow

```
New endpoint /lcl/setup
```

- ✓ Transacción atómica
- ✓ Rollback posible si algo falla
- ✓ Una sola respuesta de éxito/error

Evolución, Versionado y Flujo de Trabajo



Dolor del versionado

- Headers vs URLs
- Versionar por endpoint frente a API completa.

```
Versión header HTTP:  
GET /usuarios  
Accept-Version: v2
```

Versión URL HTTP, con versionados:

✗ Versionado global (problemático):

```
/v1/usuarios    → /v2/usuarios (sin cambios reales)  
/v1/productos   → /v2/productos (sin cambios reales)  
/v1/pedidos     → /v2/pedidos (¡AQUÍ estaba el cambio!)
```

✓ Versionado por endpoint (más limpio):

```
/v1/usuarios    (sin tocar)  
/v1/productos   (sin tocar)  
/v2/pedidos     (solo este cambió)
```

Estrategias de versionado

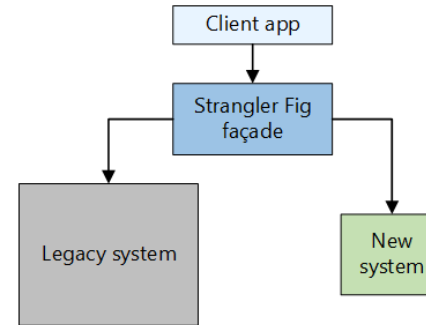
- **Sistema tradicional (v1, v2, v3)**
- **Stripe**, sistema de versionado por fecha (**27-03-2026**)
- **Solución:**
 - **Evitar versionar**
 - **Cambios pequeños**
 - **Acumular cambios críticos**

The Stripe logo, consisting of the word "stripe" in a dark blue, lowercase, sans-serif font.

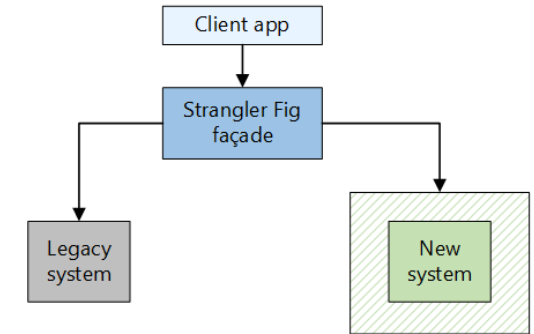
Mantenimiento de versiones

- ¿Cuánto tiempo mantengo la versión anterior funcionando?
 - Tiempo largo de deprecación
 - Brownouts
- Patrón Adapter (v1, v2, etc.)
- Strangler Fig
 - Tener dos bases en paralelo, legacy y la nueva.

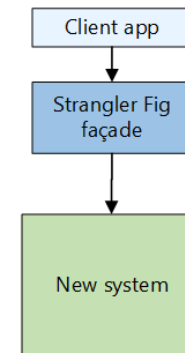
1 A façade routes client requests between the legacy system and the new system



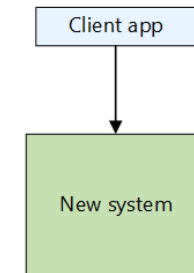
2 Incremental decomposition shifts functionality from the legacy system to the new system



3 The legacy system is fully decommissioned and has no dependencies



4 The façade is removed, and the client interacts directly with the new system



Abstracciones y el mínimo común denominador

- **Fog**, gema de Ruby para trabajar con proveedores de la nube.
- Patrón **Adapter** para **interoperabilidad**
- **Problema del mínimo común denominador**
 - Es **complicado** crear una **interfaz genérica** sin perder funcionalidad específica.
- **Evolución y Diferenciación**, batalla interminable con las empresas.

fog 1.38.0

The Ruby cloud services library. Supports all major cloud providers including AWS, Rackspace, Linode, Blue Box, StormOnDemand, and many others. Full support for most AWS services including EC2, S3, CloudWatch, SimpleDB, ELB, and RDS.

 Star 4,300

TOTAL DE DESCARGAS
59.261.670

OpenAPI

- OpenAPI es una especificación que documenta una API.
- Con el tiempo pasó a ser mucho más que eso, es un lenguaje leíble por humanos y máquinas
- Permite la generación de **código**, **tests** y **mocks**, con herramientas como **OpenAPI Generator**.

```
openapi: 3.0.0
info:
  title: API de Usuarios
  version: 1.0.0

paths:
  /usuarios/{id}:
    get:
      summary: Obtener un usuario por ID
      parameters:
        - name: id
          in: path
          required: true
          schema: [...]
```

Flujo de trabajo

